МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ» (ДГТУ)**

Факультет
Информатика и вычислительная техника

Кафедра
Программное обеспечение вычислительной техники и автоматизированных систем

# СТАТИЧЕСКИЕ ПОЛЯ И МЕТОДЫ. ДРУЖЕСТВЕННЫЕ ФУНКЦИИ

## Практикум

по выполнению лабораторной работы №2
по дисциплине «Программирование»

Программное обеспечение вычислительной техники и автоматизированных систем
*(указывается направленность (профиль) образовательной программы)*

09.03.04 Программная инженерия
*(указывается код и наименование направления подготовки)*

Ростов-на-Дону
2024 г.

Составители: канд. техн. наук, доц. каф. А.А. Скляренко

УДК 004.432

Введение в ООП на С++: метод. указания. – Ростов н/Д: Издательский центр ДГТУ, 2024.

В методическом указании рассматриваются понятия статический полей и методов, их особенности и варианты использования, также вводится понятие дружественных функций. Приведены задания к лабораторной работе, помогающие закрепить на практике полученные знания, и контрольные вопросы для самопроверки.
Предназначено для обучающихся по направлению 09.03.04 «Программная инженерия», профиль «Программное обеспечение вычислительной техники и автоматизированных систем».

Ответственный за выпуск:
зав. кафедрой «Программное обеспечение вычислительной техники и автоматизированных систем» Долгов В.В.

© А.А. Скляренко, 2024
© Издательский центр ДГТУ, 2024

# 1 Теоретическая часть

## 1.1 Статические поля и методы

Если вы объявляете переменную статической, то может существовать только одна копия этой переменной — независимо от того, сколько объектов данного класса создается. Каждый объект просто использует (совместно с другими) эту одну переменную. Для обычных переменных-членов при создании каждого объекта создается их новая копия, и доступ к каждой копии возможен только через этот объект. Для обычных переменных каждый объект обладает собственными копиями переменных. А копия статической переменной — только одна, и все объекты класса используют её совместно.

Кроме этого, одна и та же статическая переменная будет использоваться всеми классами, производными от класса, в котором эта статическая переменная содержится.

Основной смысл поддержки в C++ статических переменных-членов состоит в том, что благодаря им отпадает необходимость в использовании глобальных переменных.

Пример:

```
class A{
    static int counter;
    int my_number;
public:
    A()
    {
        my_number = counter;
        counter++;
    }
    int get_number() const
    {
        return my_number;
    }
};

int A::counter = 0;
```

Статическими можно объявлять и методы. При этом такой метод не может обращаться к не статическим полям данного класса и не имеет в себе определённым указатель this, так как вызывается независимо от экземпляра.

Статические методы могут оказаться удобными для предварительной инициализации закрытых статических атрибутов до создания экземпляровили просто независимо от доступа к ним:

```
class Rabbit{
    static int max_Rabbit_size;
    int m_size;
public:
    Rabbit():m_size(max_Rabbit_size)
    {}
    static void setDefaultRabbitSize(int new_size)
    {
        //отсюда нет доступа к «экземплярному» атрибуту m_size;
        //отсюда нет доступа к this;
        max_Rabbit_size = new_size;
    }
    int getSize() const
    {
        return m_size;
    }
};
int Rabbit::max_Rabbit_size = 10;
```

**Дружественные функции.** Это функции, которые не являются членами класса, однако имеют доступ к его закрытым членам - переменным и функциям, которые имеют спецификатор private.

Для определения дружественных функций используется ключевое слово friend.

```
class Auto
{
    friend void drive(Auto &);
    friend void setPrice(Auto &, int price);
public:
    Auto(std::string autoName, int autoPrice)
```

```
    {
        name = autoName;
        price = autoPrice;
    }
    std::string getName(){ return name; }
    int getPrice() { return price; }

private:
    std::string name;   // название автомобиля
    int price;  // цена автомобиля
};

void drive(Auto &a)
{
    std::cout << a.name << " is driven" << std::endl;
}
```

Дружественные функции могут определяться в другом классе.

## 2 Задание к лабораторной работе

Создать класс согласно варианту (таблица 1). Класс должен содержать поля, конструктор, методы (которые указаны в варианте, также можно реализовать собственные методы). Скрыть поля и часть методов. Продемонстрировать работу класса.

Добавить в класс счетчик созданных объектов на основе статического поля. Напишите программу-тест, в которой проверяется, сколько объектов класса создано при входе в функцию main, после статического создания массива объектов, после динамического создания объекта, после удаления динамического объекта.

Создайте класс с закрытыми конструкторами и деструктором. Реализуйте методы для создания и уничтожения объектов класса. Напишите программу-тест.

Создайте класс с закрытыми конструкторами и деструктором. Реализуйте

внешние функции для создания и уничтожения объектов класса. Напишите программу-тест.

Реализовать для класса дружественную функцию и продемонстрировать принцип работы с ней.

Таблица 1. Варианты заданий

| № вар. | Класс | Поля | Методы |
|---|---|---|---|
| 1. | Dog | Dog *Mparent, *Fparent;<br>string color; | changeColor( string col);<br>//сменить цвет<br>mongrelify();<br>//удалить данные о родителях |
| 2. | Patient | char * diagnose;<br>int age; | checkOut(); //выписаться<br>Diagnose(char*);<br>//Поставить диагноз |
| 3. | Wine | double alcContent,<br>density; | age(); //выдержать<br>dilute(double ratio);<br>//разбавить |
| 4. | Student | имя – char*<br>курс – int<br>пол – int(bool) | Реализовать методы: отчислить студента, зачислить студента |
| 5. | Car | double horsepower;<br>char * numplate; | tune(); //настроить двигатель<br>changeEngine (double hp);<br>//поменять двигатель |
| 6. | Exam | имя студента – char*<br>дата – int<br>оценка – int | Реализовать методы: изменить дату/оценку, добавить новый экзамен. |
| 7. | Animal | имя – char*<br>класс – char*<br>средний вес – int | Реализовать методы: добавить новое животное, изменить средний вес. |
| 8. | Fruit | double sweetness,<br>bitterness, sourness;<br>string name; | juice(double sugar);<br>plant(); |
| 9. | Smartphone | int traficSpent;<br>int traficPlan;<br>int trafficPlanCost;<br>int price; | download (int size); //скачать что-то<br>nextmonth();<br>//наступает след месяц |
| 10. | Notebook | string *contents;<br>int size; | note (string nt); //записать<br>seal(); //запечатать |
| 11. | Shop | имя – char*<br>начальник – char*<br>количество работающих – int | Реализовать методы: изменить количество работающих в конкретном магазине, добавить новый магазин. |
| 12. | Ship | имя – char*<br>водоизмещение – int<br>тип – char* | Реализовать методы: построить новую башню, изменить характеристики существующей |

| | | | |
|---|---|---|---|
| 13. | Product | имя – char*<br>количество – int<br>стоимость – float | Реализовать методы: увеличить/уменьшить количество или стоимость продуктов. |
| 14. | Receipt | номер – int<br>дата – int<br>сумма – float | Реализовать методы: изменить стоимость рецепта. |
| 15. | Building | char* address;<br>int storeyCount; | addStoreys(int count); //добавить этажей<br>quarantine(); //закрыть на карантин |
| 16. | Product factories | имя – char*<br>шифр – char*<br>количество – int | Реализовать методы: увеличить продукцию, изменить шифр конкретного продукта. |
| 17. | Library | имя – char*<br>автор – char*<br>стоимость – float | Реализовать методы: изменение стоимости, добавление новой книги. |
| 18. | Bow | double strength;<br>string BowStringMat; | restring(string bsm); // поменять тетиву<br>test(); |
| 19. | Address | имя – char*<br>улица – char*<br>номер дома – int | Реализовать методы: отправить посылку, получить посылку |
| 20. | Country | имя – char*<br>форма<br>правления – char*<br>площадь – float | Реализовать методы: изменение численности населения. |
| 21. | Personnel | имя – char*<br>номер цеха – int<br>разряд – int | Реализовать методы: добавить сотрудника, удалить сотрудника |
| 22. | Automobile | марка – char*<br>мощность – int<br>стоимось – float | Реализовать методы: купить автомобиль, продать автомобиль. |
| 23. | Missile | double speed;<br>string sender; | launch(double spd); //запустить<br>land();<br>//попасть |
| 24. | Clerk | имя – char*<br>возраст – int<br>рабочий стаж – int | Реализовать методы: изменить возраст, изменить стаж. |
| 25. | Person | имя – char*<br>возраст – int<br>пол – int(bool) | Реализовать методы: изменить возраст конкретного человека, добавить нового человека |

## 3 Материально-техническое обеспечение работы

Аудитория для проведения лабораторных занятий должна быть укомплектована специализированной мебелью и индивидуальными компьютерами следующей минимальной комплектации:

- Процессор: не менее двух исполнительных ядер, совместимый с системой команд x86 и x64, с поддержкой аппаратной виртуализации.
- Оперативная память: не менее 8 Гб.
- Монитор: не менее 24" (дюймов) по диагонали.
- Наличие локальной сети со скоростью обмена не менее 1 Гб/сек.
- Наличие доступа в сеть Интернет со скоростью не менее 1 Мбит/сек.
- Наличие клавиатуры и манипулятора «мышь».

На компьютерах должно быть установлено следующее программное обеспечение:

- Операционная система: Microsoft Windows 10 (или выше) и/или одна из операционных систем семейства Linux, допускающих установку среды исполнения для C++. Допускается конфигурация, когда одна из операционных систем установлена внутри виртуальной машины (гипервизора).
- Среды разработки и исполнения программ на C++.

## 4 Порядок выполнения и сдачи работы

Для выполнения лабораторной работы рекомендуется придерживаться следующего порядка выполнения:

1. Ознакомится с темой и целями лабораторной работы
2. Изучить теоретический материал
3. Подготовить рабочее окружение
4. Разработать программный код лабораторной работы в соответствие с заданием
5. Произвести проверку работоспособности, выполнить тестирование и отладку кода
6. Проанализировать полученные результаты и примененные в ходе решения подходы
7. Выполнить самооценку и рефлексию
8. Сдать лабораторную работу преподавателю и получить от него обратную связь

При сдаче студентом лабораторной работы основным отчетом выступает исходный код самостоятельно созданной в процессе выполнения работы программы.

Исходный код должен быть отформатирован согласно принятым для используемого языка программирования стандартам. Является желательным наличие в исходном коде комментариев, описывающих основные части программы и особенности их функционирования. В то же время студент должен быть готов объяснить работу программы в целом и каждую отдельную ее часть при полном отсутствии комментариев (например, они могут быть удалены преподавателем при сдаче работы).

Обязательным условием сдачи является умение студента самостоятельно восстановить любой участок исходного кода программы (но не более 20 строк подряд) после его удаления. Удаленный участок должен быть самостоятельно набран студентом заново в присутствие преподавателя. При восстановлении участка запрещается использовать операции Undo (Отменить) текстовых редакторов или переписывание кода участка из других источников.

Исходный код сдаваемой программы должен быть представлен в электронном виде.

Сдача исходного кода ранее сдававшихся программ или программ, код которых выложен в сети Интернет, не допускается.

## 5 Контрольные вопросы к лабораторной работе

1. Что такое ООП (объектно-ориентированное программирование) и в чем его основные принципы?

2. Какие основные понятия используются в ООП, такие как класс, объект, наследование, инкапсуляция и полиморфизм?

3. Что такое класс и как он определяется?

4. Как создать объект класса и каким образом можно обращаться к его полям и методам?

5. Что такое статическое поле?

6. Что  такое статический метод?

7. Что такое дружественная функция?

## 6 Перечень использованных информационных ресурсов

1. Ноткин, А. М. Объектно-ориентированное программирование: ООП на языке C++ : учебное пособие / А. М. Ноткин. — Пермь : ПНИПУ, 2013. — 230 с. — ISBN 978-5-398-00966-8. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/160806 (дата обращения: 22.08.2024). — Режим доступа: для авториз. пользователей.

2. Скворцова, Л. А. Объектно-ориентированное программирование на языке C++ : учебное пособие / Л. А. Скворцова. — Москва : РТУ МИРЭА, 2020. — 246 с. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/163862 (дата обращения: 22.08.2024). — Режим доступа: для авториз. пользователей.

Редактор А.А. Литвинова

---

| | | |
|---|---|---|
| ЛР № 04779 от 18.05.01. | В набор | В печать |
| Объем 0,5 усл.п.л., уч.-изд.л. | Офсет. | Формат 60x84/16. |
| Бумага тип №3. | Заказ № | Тираж 75. Цена |

---

Издательский центр ДГТУ

Адрес университета и полиграфического предприятия:

344010, г. Ростов-на-Дону, пл. Гагарина, 1.